

# **Normalization**

**1 2 3 NF**

# Functional Dependencies

$\alpha \rightarrow \beta$  is trivial if  $\beta \subseteq \alpha$

## Functional Dependency

$A \rightarrow C??$

$C \rightarrow A ??$

| <i>A</i>              | <i>B</i>              | <i>C</i>              | <i>D</i>              |
|-----------------------|-----------------------|-----------------------|-----------------------|
| <i>a</i> <sub>1</sub> | <i>b</i> <sub>1</sub> | <i>c</i> <sub>1</sub> | <i>d</i> <sub>1</sub> |
| <i>a</i> <sub>1</sub> | <i>b</i> <sub>2</sub> | <i>c</i> <sub>1</sub> | <i>d</i> <sub>2</sub> |
| <i>a</i> <sub>2</sub> | <i>b</i> <sub>2</sub> | <i>c</i> <sub>2</sub> | <i>d</i> <sub>2</sub> |
| <i>a</i> <sub>2</sub> | <i>b</i> <sub>3</sub> | <i>c</i> <sub>2</sub> | <i>d</i> <sub>3</sub> |
| <i>a</i> <sub>3</sub> | <i>b</i> <sub>3</sub> | <i>c</i> <sub>2</sub> | <i>d</i> <sub>4</sub> |

# 1NF Summarized

- **Each attribute must be atomic (single value)**
  - No repeating columns within a row (composite attributes)
  - No multi-valued columns.

All key attributes defined

All attributes dependent on primary key

- **1NF simplifies attributes**
  - Queries become easier.

# Dependencies and Normal Form

- The schema R is in 1NF if domain of all attributes of R is atomic
- Desirable
  - PK dependencies
- Undesirable
  - Partial Key : based on part of composite PK
  - Transitive: one nonprime attribute depends on another nonprime attribute

# Normal Forms

- Scalar Values
  - 1 NF
- Normal Forms based on PK
  - 2 NF
  - 3 NF
- Normal Forms based on CKs
  - Boyce-Codd Normal Form (BCNF)
- Other Normal Forms
  - 4 NF (Multivalued Dependencies)
  - 5 NF (Join Dependencies)
  - Deal with very rare practical situations

# Normal Forms

- 1NF : A relation is in a first normal form, if every tuple contains exactly one value for each attribute
- Example: Suppliers1 ( **S#** , status, city, **P#** , qty)
- Primary key ( S#, P#)
- City  $\rightarrow$  status (FD)
- FD Diagram (FDD) can be drawn
- Observe
  - Status depends on the city (FD)
  - City depends only on the supplier number

## Redundancies in Suppliers1

- **Insert:** We cannot insert the fact that a particular supplier is located in the city unless he/she actually supplies some part
- The Suppliers1 may not show a supplier is located in Chennai
- **Delete:** If we delete a sole Suppliers1 tuple for a particular supplier, we also delete that he/she is located in a particular city
- **Update:** to update the city for a particular supplier

# Schema Refinement for Suppliers1

- Refined Schema
  - Suppliers2 ( S#, status, city)
  - Suppliers\_Parts ( S#, P#, qty)
- Insert: We can insert the info that s-5 is located in Pune although s-5 does not currently supply any part
- Delete: We do not lose the info that s-18 is located in Surat even if all the s-18 tuples from the suppliers\_parts are deleted
- Update: city for a supplier can be updated easily through suppliers2

## Second Normal Form 2NF

- A relation is in 2NF only if it is in 1NF and every nonkey attribute is irreducibly dependent on the primary key.  
(here, we are assuming only single candidate ( hence primary ) key case)
- The original relation ***Suppliers1*** can be converted to the 2NF form by taking projection of it to a set of 2 relations ***suppliers2*** and ***suppliers\_parts***
- All relations with single attribute PK are in 2 NF!!
- 2NF applies to relations with composite keys

## 2 NF

- A relation that is in 1NF & every non-PK attribute is fully dependent on the PK, is said to be in 2 NF



## 2NF Problems: Suppliers2

- Insert: we cannot insert the fact that a particular city has a particular status unless we have some supplier located in that city
- Delete: if we delete a sole tuple for a particular city we delete the information for the supplier and also the information that a city has a particular status
- Update: the status for a city appears many a times in the suppliers2. The change in the Mumbai status from 80 to 90 may need changes in 100 tuples

# Schema Refinement for Suppliers2

- Replace Suppliers2 by
  - Suppliers3 ( S#, city )
  - City\_Info ( city, status)
- **3NF** : A relation is in a 3NF if it is in 2NF and every nonkey attribute is nottransitively dependent on the primary key
- A relation is in a 3NF if nonkey attributes are:
  - Mutually independent
  - Irreducibly/nontransitively dependent on the primary key
- A nonkey attribute is any attribute that does not participate in the primary key of that relation

## 3 NF

- A relation that is in 1NF & 2 NF & no non-PK attribute is transitively dependent on the PK, is said to be in 3 NF



# Heath's Theorem

- $R(A,B,C)$  where  $A$   $B$   $C$  are sets of attributes
- If  $R$  has FD  $A \twoheadrightarrow B$ , then  $R$  equals join of  $\{A\ B\}$  and  $\{A\ C\}$

- Example:

**$S ( \underline{s\#}, status, city ) ;$**

PK:  $s\#$  implies  $s\# \twoheadrightarrow city$  and  $s\# \twoheadrightarrow status$

FD:  $City \twoheadrightarrow status$  is troublesome ( out of non-candidate key)

Then decomposition

$(s\#,city), (status, city)$  is **Dependency Preserving**

# Heath's Theorem

- it says that a relation  $R$  over an attribute set  $U$  and satisfying a functional dependency  $X \rightarrow Y$  can be safely split in two relations having the lossless-join decomposition property, namely into

$$\pi_{XY}(R) \bowtie \pi_{XZ}(R) = R$$

- where  $Z = U - XY$  are the rest of the attributes.
- functional dependency provides a simple way to construct a lossless-join decomposition of  $R$  in two smaller relations

# Normal Forms

- Returning to the issue of schema refinement, the first question to be asked is whether any refinement is needed!
- If a relation is in a certain *normal form* (BCNF, 3NF etc.), it is known that certain kinds of problems are avoided/minimized. This can be used to help us decide whether decomposing the relation will help.
- Role of FDs in detecting redundancy:
  - Consider a relation R with 3 attributes, ABC.
    - No FDs hold: There is no redundancy here.
    - Given  $A \rightarrow B$ : Several tuples could have the same A value, and if so, they'll all have the same B value!

# Boyce-Codd Normal Form BCNF (1974)

- 2 or more candidate keys
  - Composite candidate keys
  - Overlapped keys
- 
- A relation is in BCNF if and only if only determinants are candidate keys

# BCNF

- Based on FDs that take into account all candidate keys of a relation
- For a relation with only 1 CK, 3NF & BCNF are equivalent
- A relation is said to be in BCNF if every determinant is a CK

# BCNF

- Relation Suppliers1 ( **s#**, status, city, **p#**, qty) is not in BCNF. It has 3 determinants s#, city and (s#,p#)
- Relation Suppliers2( s#, status, city) is not in BCNF. It has s# and city as determinants
- But SP (s#,p#,qty), SC (s#,city) and CS ( city,status) are in BCNF

## Contd...

- Nonoverlapping candidate keys
  - S( s#,sname,city ,status)
  - S# and sname are candidate keys and city->status no longer holds
  - S is in BCNF
- Overlapping candidate keys
  - SSP( s#,sname, p#,qty) is not in BCNF
  - S#->sname and viceversa prevents it to be in BCNF
  - Redudndancies: same supplier,name pair repeated for various parts that the supplier is supplying
  - SS(s#,sname) , SP( s#,p#,qty)

# Decomposition of a Relation Scheme

- Suppose that relation  $R$  contains attributes  $A_1 \dots A_n$ . A decomposition of  $R$  consists of replacing  $R$  by two or more relations such that:
- Each new relation scheme contains a subset of the attributes of  $R$  (and no attributes that do not appear in  $R$ ), and
  - Every attribute of  $R$  appears as an attribute of one of the new relations.
- Intuitively, decomposing  $R$  means we will store instances of the relation schemes produced by the decomposition, instead of instances of  $R$ .
- E.g., Can decompose SNLRWH into SNLRH and RW.

## Example Decomposition

- Decompositions should be used only when needed.
  - SNLRWH has FDs  $S \rightarrow \text{SNLRWH}$  and  $R \rightarrow W$
  - Second FD causes violation of 3NF; W values repeatedly associated with R values. Easiest way to fix this is to create a relation RW to store/preserve these associations, and to remove W from the main schema:
    - i.e., we decompose SNLRWH into SNLRH and RW
- The information to be stored consists of SNLRWH tuples. If we just store the projections of these tuples onto SNLRH and RW, are there any potential problems that we should be aware of?

# Lossless Decomposition

## *Theorem*

A decomposition of  $R$  into  $R1$  and  $R2$  is lossless join wrt FDs  $F$ , if and only if at least one of the following dependencies is in  $F^+$ :

- $R1 \cap R2 \rightarrow R1$
- $R1 \cap R2 \rightarrow R2$

In other words,  $R1 \cap R2$  forms a superkey of either  $R1$  or  $R2$

- *Optionally you can infer about a decomposition by actually checking joins of the decomposed relation instances*

# Boyce-Codd Normal Form

- A relation schema  $R$  is in BCNF with respect to a set  $F$  of functional dependencies if for all functional dependencies in  $F^+$  of the form

$$\alpha \rightarrow \beta$$

where  $\alpha \subseteq R$  and  $\beta \subseteq R$ , at least one of the following holds:

- $\alpha \rightarrow \beta$  is trivial (i.e.,  $\beta \subseteq \alpha$ )
- $\alpha$  is a superkey for  $R$

## Boyce-Codd Normal Form (Cont.)

- Example schema that is **not** in BCNF:

*in\_dep (ID, name, salary, dept\_name, building, budget )*

because :

- *dept\_name* → *building, budget*

- holds on *in\_dep*

*but*

- *dept\_name* is not a superkey

- When decompose *in\_dept* into *instructor* and *department*

- *instructor* is in BCNF

- *department* is in BCNF

# Decomposing a Schema into BCNF

- Let  $R$  be a schema  $R$  that is not in BCNF. Let  $\alpha \rightarrow \beta$  (nontrivial where  $\alpha$  is not a superkey) be the FD that causes a violation of BCNF.
- We decompose  $R$  into:
  - $(\alpha \cup \beta)$
  - $(R - (\beta - \alpha))$
- In our example of *in\_dep*,
  - $\alpha = \text{dept\_name}$
  - $\beta = \text{building, budget}$and *in\_dep* is replaced by
  - $(\alpha \cup \beta) = (\text{dept\_name}, \text{building}, \text{budget})$
  - $(R - (\beta - \alpha)) = (\text{ID}, \text{name}, \text{dept\_name}, \text{salary})$

# Example

- $R = (A, B, C)$   
 $F = \{A \rightarrow B, B \rightarrow C\}$
- $R_1 = (A, B), R_2 = (B, C)$ 
  - Lossless-join decomposition:  
$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow BC$$
  - Dependency preserving
- $R_1 = (A, \underline{B}), R_2 = (A, C)$ 
  - Lossless-join decomposition:  
$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$
  - Not dependency preserving  
(cannot check  $B \rightarrow C$  without computing  $R_1 \bowtie R_2$ )

# BCNF and Dependency Preservation

- database consistency constraints: primary-key constraints, functional dependencies, check constraints, assertions, and triggers.
- It is not always possible to achieve both BCNF and dependency preservation
- Consider a schema:  
*dept\_advisor(s\_ID, i\_ID, department\_name)*
- With function dependencies:  
 $i\_ID \rightarrow dept\_name$   
 $s\_ID, dept\_name \rightarrow i\_ID$
- *dept\_advisor* is not in BCNF
  - *i\_ID* is not a superkey.
- Any decomposition of *dept\_advisor* will not include all the attributes in  
 $s\_ID, dept\_name \rightarrow i\_ID$
- Thus, the composition is NOT be dependency preserving

# Dependency Preservation

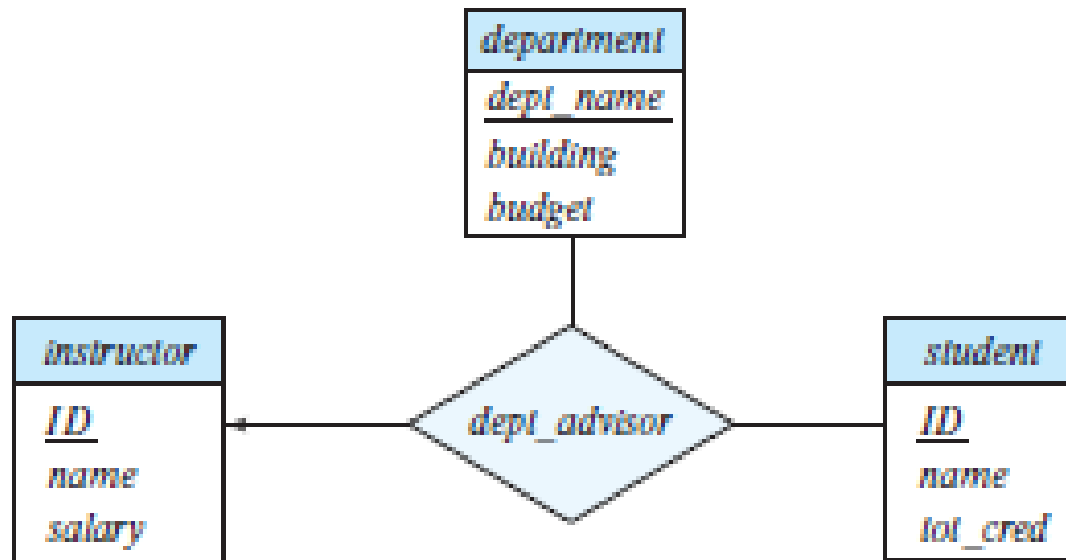


Figure 7.6 The *dept\_advisor* relationship set.

# Third Normal Form

- A relation schema  $R$  is in **third normal form (3NF)** if for all:

$$\alpha \rightarrow \beta \text{ in } F^+$$

at least one of the following holds:

- $\alpha \rightarrow \beta$  is trivial (i.e.,  $\beta \in \alpha$ )
- $\alpha$  is a superkey for  $R$
- Each attribute  $A$  in  $\beta - \alpha$  is contained in a candidate key for  $R$ .

**(NOTE:** each attribute may be in a different candidate key)

- If a relation is in BCNF it is in 3NF (since in BCNF one of the first two conditions above must hold).
- Third condition is a minimal relaxation of BCNF to ensure dependency preservation (will see why later).

# 3NF Example

- Consider a schema:

*dept\_advisor(s\_ID, i\_ID, dept\_name)*

- With function dependencies:

$i\_ID \rightarrow dept\_name$

$s\_ID, dept\_name \rightarrow i\_ID$

- Two candidate keys =  $\{s\_ID, dept\_name\}, \{s\_ID, i\_ID\}$
- We have seen before that *dept\_advisor* is **not** in BCNF
- R*, however, is in 3NF
  - $s\_ID, dept\_name$  is a superkey
  - $i\_ID \rightarrow dept\_name$  and  $i\_ID$  is NOT a superkey, but:
    - $\{dept\_name\} - \{i\_ID\} = \{dept\_name\}$  and
    - $dept\_name$  is contained in a candidate key

# Redundancy in 3NF

- Consider the schema  $R$  below, which is in 3NF
  - $R = (J, K, L)$
  - $F = \{JK \rightarrow L, L \rightarrow K\}$
  - And an instance table:

| $J$    | $L$   | $K$   |
|--------|-------|-------|
| $j_1$  | $l_1$ | $k_1$ |
| $j_2$  | $l_1$ | $k_1$ |
| $j_3$  | $l_1$ | $k_1$ |
| $null$ | $l_2$ | $k_2$ |

- What is wrong with the table?
  - Repetition of information
  - Need to use null values (e.g., to represent the relationship  $l_2, k_2$  where there is no corresponding value for  $J$ )

# Comparison of BCNF and 3NF

- Advantages to 3NF over BCNF. It is always possible to obtain a 3NF design without sacrificing losslessness or dependency preservation.
- Disadvantages to 3NF.
  - We may have to use null values to represent some of the possible meaningful relationships among data items.
  - problem of repetition of information.

# Goals of Normalization

- Let  $R$  be a relation scheme with a set  $F$  of functional dependencies.
- Decide whether a relation scheme  $R$  is in “good” form.
- In the case that a relation scheme  $R$  is not in “good” form, need to decompose it into a set of relation scheme  $\{R_1, R_2, \dots, R_n\}$  such that:
  - Each relation scheme is in good form
  - The decomposition is a lossless decomposition
  - Preferably, the decomposition should be dependency preserving.

# How good is BCNF?

- There are database schemas in BCNF that do not seem to be sufficiently normalized
- Consider a relation  
*inst\_info* (*ID*, *child\_name*, *phone*)
  - where an instructor may have more than one phone and can have multiple children
  - Instance of *inst\_info*

| <i>ID</i> | <i>child_name</i> | <i>phone</i> |
|-----------|-------------------|--------------|
| 99999     | David             | 512-555-1234 |
| 99999     | David             | 512-555-4321 |
| 99999     | William           | 512-555-1234 |
| 99999     | William           | 512-555-4321 |

## How good is BCNF? (Cont.)

- There are no non-trivial functional dependencies and therefore the relation is in BCNF
- Insertion anomalies – i.e., if we add a phone 981-992-3443 to 99999, we need to add two tuples

(99999, David, 981-992-3443)

(99999, William, 981-992-3443)

# Higher Normal Forms

- It is better to decompose *inst\_info* into:
  - *inst\_child*:

| <i>ID</i> | <i>child_name</i> |
|-----------|-------------------|
| 99999     | David             |
| 99999     | William           |

- *inst\_phone*:

| <i>ID</i> | <i>phone</i> |
|-----------|--------------|
| 99999     | 512-555-1234 |
| 99999     | 512-555-4321 |

- This suggests the need for higher normal forms, such as Fourth Normal Form (4NF), which we shall see later